



UNIT 5 - it describe about guifelines for machine learning experiments

Machine Learning (Anna University)



Scan to open on Studocu

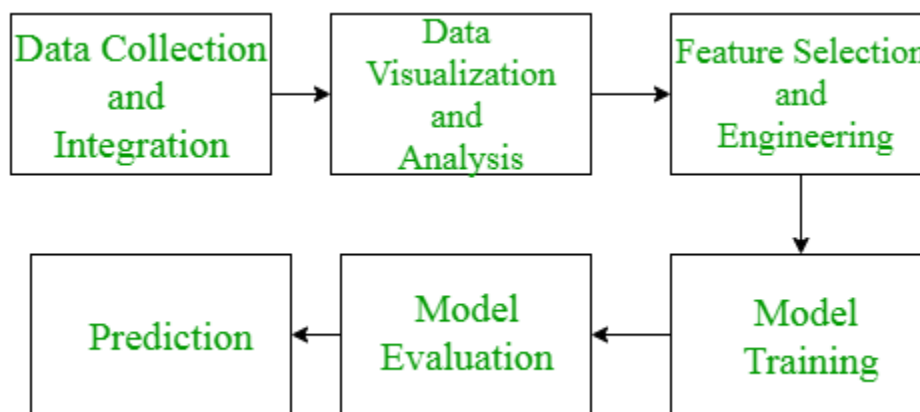
UNIT 5

Guidelines for machine learning experiments, Cross Validation (CV) and resampling – Kfold CV, bootstrapping, measuring classifier performance, assessing a single classification algorithm and comparing two classification algorithms – t test, Mc Nemar's test, K-fold CV paired t test

GUIDELINES FOR MACHINE LEARNING EXPERIMENTS

- The Machine Learning pipeline starts with data collection and integration. After data is collected analysis and visualization of data is done. Further, the most crucial step feature selection and engineering is performed then the model is trained. After that model, evaluation is done and our model becomes ready for prediction!
- To understand the pipeline well, consider building an ML model for a company's customer care service. Consider a company XYZ, as an online book shop, that delivers books and kindle to its customer and this company wants an improved customer care service. It wants that if a customer calls the helpline for any kind of issue, say for replacement of books, complaint of kindle purchased or some other services. The company wants to ensure that the customer's call gets directed to the right service person in minimum time and that the process should be smooth. To build a model for customer care services of the company, we will use the ML pipeline for the systematic development of the model.

Here is the diagrammatic view of the ML pipeline:



MACHINE LEARNING PIPELINE

1. Data Collection and integration:

- The first step of the ML pipeline involves the collection of data and integration of data.
- Data collected acts as an input to the model (data preparation phase)
- Inputs are called features.

- Data collected in the case of our considered example involves a lot of data. The collected data should answer the following questions- What is past customer history? What were the past orders? Is the customer a prime member of our bookstore? Does the customer own a kindle? Has the customer made any previous complaints? What was the most number of complaints?
- The more the data is, more the better our model becomes.
- Once the data is collected we need to integrate and prepare the data.
- Integration of data means placing all related data together.
- Then data preparation phase starts in which we manually and critically explore the data.
- The data preparation phase tells the developer that is the data matching the expectations. Is there enough info to make an accurate prediction? Is the data consistent?

2. Exploratory Data Analysis and Visualisation:

- Once the data is prepared developer needs to visualize the data to have a better understanding of relationships within the dataset.
- When we get to see data, we can notice the unseen patterns that we may not have noticed in the first phase.
- It helps developers easily identify missing data and outliers.
- Data visualization can be done by plotting histograms, scatter plots, etc.
- After visualization is done data is analyzed so that developer can decide what ML technique he may use.
- In the considered example case unsupervised learning may be used to analyze customer purchasing habits.

3. Feature Selection and Engineering:

- Feature selection means selecting what features the developer wants to use within the model.
- Features should be selected so that a minimum correlation exists between them and a maximum correlation exists between the selected features and output.
- Feature engineering is the process to manipulate the original data into new and potential data that has a lot many features within it.
- In simple words Feature engineering is converting raw data into useful data or getting the maximum out of the original data.
- Feature engineering is arguably the most crucial and time-consuming step of the ML pipeline.
- Feature selection and engineering answers questions – Are these features going to make any sense in our prediction?
- It deals with the accuracy and precision of data.

4. Model Training:

- After the first three steps are done completely we enter the model training phase.
- It is the first step officially when the developer gets to train the model on basis of data.
- To train the model, data is split into three parts- Training data, validation data, and test data.
- Around 70%-80% of data goes into the training data set which is used in training the model.
- Validation data is also known as development set or dev set and is used to avoid overfitting or underfitting situations i.e. enabling hyperparameter tuning.
- Hyperparameter tuning is a technique used to combat overfitting and underfitting.
- Validation data is used during model evaluation.
- Around 10%-15% of data is used as validation data.
- Rest 10%-15% of data goes into the test data set. Test data set is used for testing after the model preparation.
- It is crucial to randomize data sets while splitting the data to get an accurate model.
- Data can be randomized using Scikit learn in python.

5. Model Evaluation:

- After the model training, validation, or development data is used to evaluate the model.
- To get the most accurate predictions to test data may be used for further model evaluation.
- A confusion matrix is created after model evaluation to calculate accuracy and precision numerically.
- After model evaluation, our model enters the final stage that is prediction.

6. Prediction:

- In the prediction phase developer deploys the model.
- After model deployment, it becomes ready to make predictions.
- Predictions are made on training data and test data to have a better understanding of the build model.

The deployment of the model isn't a one-time exercise. As more and more data gets generated, the model is trained on new data, evaluated again, and deployed again. Model training, model evaluation, and prediction phase circulate each other.

CROSS-VALIDATION

In **machine learning**, we couldn't fit the model on the training data and can't say that the model will work accurately for the real data. For this, we must assure that our model got the correct patterns

from the data, and it is not getting up too much noise. For this purpose, we use the **cross-validation technique**. In this article, we'll delve into the process of cross-validation in machine learning.

What is Cross-Validation?

Cross validation is a technique used in machine learning to evaluate the performance of a model on unseen data. It involves dividing the available data into multiple folds or subsets, using one of these folds as a validation set, and training the model on the remaining folds. This process is repeated multiple times, each time using a different fold as the validation set. Finally, the results from each validation step are averaged to produce a more robust estimate of the model's performance. Cross validation is an important step in the machine learning process and helps to ensure that the model selected for deployment is robust and generalizes well to new data.

What is cross-validation used for?

The main purpose of cross validation is to prevent overfitting, which occurs when a model is trained too well on the training data and performs poorly on new, unseen data. By evaluating the model on multiple validation sets, cross validation provides a more realistic estimate of the model's generalization performance, i.e., its ability to perform well on new, unseen data.

Types of Cross-Validation

There are several types of cross validation techniques, including **k-fold cross validation**, **leave-one-out cross validation**, and **Holdout validation**, **Stratified Cross-Validation**. The choice of technique depends on the size and nature of the data, as well as the specific requirements of the modeling problem.

1. Holdout Validation

In Holdout Validation, we perform training on the 50% of the given dataset and rest 50% is used for the testing purpose. It's a simple and quick way to evaluate a model. The major drawback of this method is that we perform training on the 50% of the dataset, it may possible that the remaining 50% of the data contains some important information which we are leaving while training our model i.e. higher bias.

2. LOOCV (Leave One Out Cross Validation)

In this method, we perform training on the whole dataset but leaves only one data-point of the available dataset and then iterates for each data-point. In LOOCV, the model is trained on $n-1$ samples and tested on the one omitted sample, repeating this process for each data point in the dataset. It has some advantages as well as disadvantages also.

An advantage of using this method is that we make use of all data points and hence it is low bias.

The major **drawback** of this method is that it leads to **higher variation** in the testing model as we are testing against one data point. If the data point is an outlier it can lead to higher variation. Another drawback is it **takes a lot of execution time** as it iterates over 'the number of data points' times.

3. Stratified Cross-Validation

It is a technique used in machine learning to ensure that each fold of the cross-validation process maintains the same class distribution as the entire dataset. This is particularly important when dealing with imbalanced datasets, where certain classes may be underrepresented. In this method,

1. The dataset is divided into k folds while maintaining the proportion of classes in each fold.
2. During each iteration, one-fold is used for testing, and the remaining folds are used for training.
3. The process is repeated k times, with each fold serving as the test set exactly once.

Stratified Cross-Validation is essential when dealing with classification problems where maintaining the balance of class distribution is crucial for the model to generalize well to unseen data.

4. K-Fold Cross Validation

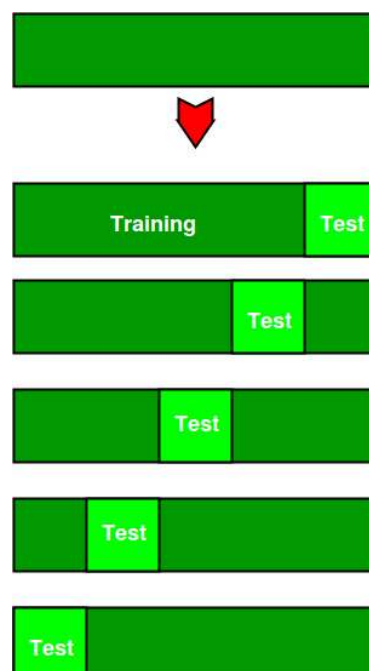
In K-Fold Cross Validation, we split the dataset into k number of subsets (known as folds) then we perform training on the all the subsets but leave one($k-1$) subset for the evaluation of the trained model. In this method, we iterate k times with a different subset reserved for testing purpose each time.

Note: It is always suggested that the value of k should be 10 as the lower value of k takes towards validation and higher value of k leads to LOOCV method.

Example of K Fold Cross Validation

The diagram below shows an example of the training subsets and evaluation subsets generated in k -fold cross-validation. Here, we have total 25 instances. In first iteration we use the first 20 percent of data for evaluation, and the remaining 80 percent for training ([1-5] testing and [5-25] training) while in the second iteration we use the second subset of 20 percent for evaluation, and the remaining three subsets of the data for training ([5-10] testing and [1-5 and 10-25]

training), and so on.



Advantages:

1. **Overcoming Overfitting:** Cross validation helps to prevent overfitting by providing a more robust estimate of the model's performance on unseen data.
2. **Model Selection:** Cross validation can be used to compare different models and select the one that performs the best on average.
3. **Hyperparameter tuning:** Cross validation can be used to optimize the hyperparameters of a model, such as the regularization parameter, by selecting the values that result in the best performance on the validation set.
4. **Data Efficient:** Cross validation allows the use of all the available data for both training and validation, making it a more data-efficient method compared to traditional validation techniques.

Disadvantages:

1. **Computationally Expensive:** Cross validation can be computationally expensive, especially when the number of folds is large or when the model is complex and requires a long time to train.
2. **Time-Consuming:** Cross validation can be time-consuming, especially when there are many hyperparameters to tune or when multiple models need to be compared.
3. **Bias-Variance Tradeoff:** The choice of the number of folds in cross validation can impact the bias-variance tradeoff, i.e., too few folds may result in high variance, while too many folds may result in high bias.

BOOTSTRAPPING

Bootstrapping

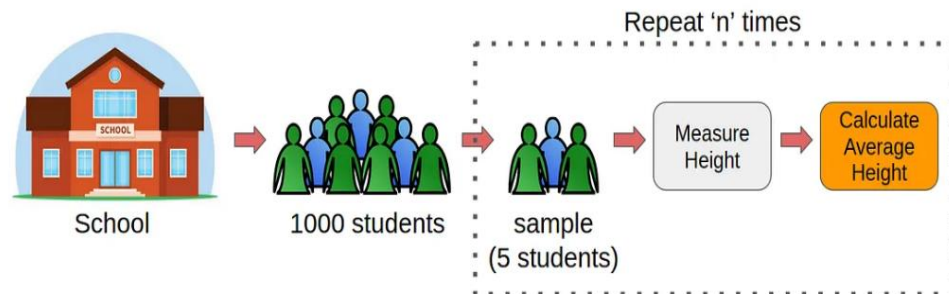
Bootstrapping is a resampling technique that involves repeatedly drawing samples from the dataset with replacement and estimating model performance on these samples. It provides a way to assess the uncertainty in the performance metrics.

How Bootstrapping Works

1. **Generate Bootstrap Samples:**
 - Draw n samples from the original dataset (with replacement) to create a bootstrap sample.
 - This process is repeated B times to create B bootstrap samples.
2. **Train and Evaluate the Model:**
 - Train the model on each bootstrap sample and evaluate it on the out-of-bag (OOB) data, which consists of data points not included in the bootstrap sample.
 - The OOB error estimate is used to gauge model performance.

3. Aggregate Results:

- Calculate the performance metrics for each bootstrap sample.
- Average the results to get an overall performance estimate.



“Instead of measuring the heights of all the students, we can draw a random sample of 5 students and measure their heights. We would repeat this process 20 times and then average the collected height data of 100 students (5×20). This average height would be an estimate of the mean height of all the students of the school”.

Advantages of Bootstrapping

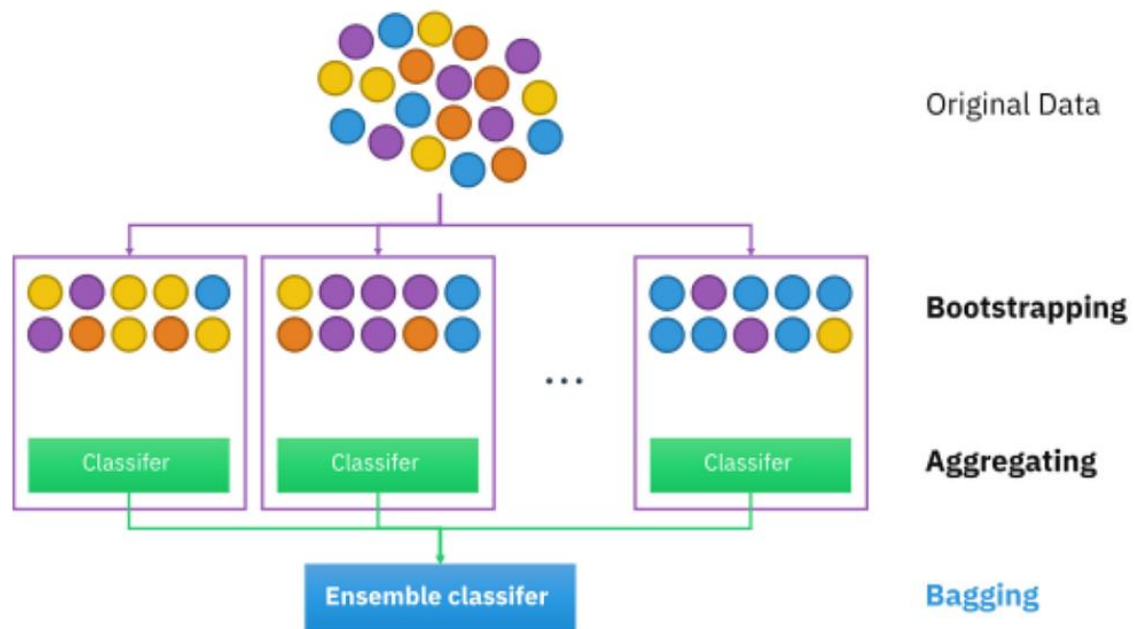
- **Flexibility:** Can be applied to small datasets where cross-validation might not be feasible.
- **Variance Estimation:** Provides an estimate of the variability of the performance metrics.
- **Bias Correction:** Helps in reducing the bias in the performance estimate.

Disadvantages of Bootstrapping

- **Overfitting:** Since bootstrap samples are drawn with replacement, they might include duplicate data points, which can lead to overfitting.
- **Computationally Demanding:** Similar to cross-validation, it requires significant computational resources, especially for a large number of bootstrap samples.

• Bootstrap Sampling in Machine Learning

- Bootstrap sampling is used in a machine learning ensemble algorithm called bootstrap aggregating (also called bagging). It helps in avoiding overfitting and improves the stability of machine learning algorithms.
- In bagging, a certain number of equally sized subsets of a dataset are extracted with replacement. Then, a machine learning algorithm is applied to each of these subsets and the outputs are ensembled as I have illustrated below:



Bootstrapping Pseudo Code

```
function bootstrap(data, model, num_samples):
    scores = []
    for i in range(num_samples):
        bootstrap_sample = draw_bootstrap_sample(data)
        model.fit(bootstrap_sample)
        score = model.evaluate(data)
        scores.append(score)
    average_score = average(scores)
    return average_score
```

Key Differences Between Cross-Validation and Bootstrapping

While cross-validation and bootstrapping share similarities in their purpose and applications, they differ significantly in their methodologies, advantages, and ideal use cases. Understanding these differences is crucial for selecting the appropriate technique for a given task.

1. Data Partitioning

- **Cross-Validation:** The dataset is split into mutually exclusive subsets or folds. Each fold is used once as the test set while the remaining folds are used for training.
 - Example: In k-fold cross-validation, the dataset is split into k parts. The model is trained on k-1 of these parts and tested on the remaining part. This process is repeated k times, with each part used as the test set once.

- **Bootstrapping:** Involves sampling with replacement to create multiple bootstrap samples from the original dataset. Each sample may contain duplicate data points.
 - Example: A bootstrap sample of size n is created by randomly drawing n data points with replacement from the original dataset. This process is repeated B times.

2. Structure of Samples

- **Cross-Validation:** Each fold is a unique subset of the data with no overlap between training and test sets in any given iteration.
- **Bootstrapping:** Each bootstrap sample is likely to contain repeated instances of some data points and may omit others. The test set (out-of-bag data) consists of data points not included in the bootstrap sample.

3. Bias and Variance

- **Cross-Validation:** Typically provides a lower variance estimate of model performance due to averaging over multiple folds, but may have higher bias if the number of folds is small.
- **Bootstrapping:** Can provide a lower bias estimate as it uses the full dataset for each bootstrap sample but may have higher variance due to the nature of resampling with replacement.

4. Performance Estimation

- **Cross-Validation:** Typically provides a lower variance estimate of model performance due to averaging over multiple folds, but may have higher bias if the number of folds is small.
- **Bootstrapping:** Can provide a lower bias estimate as it uses the full dataset for each bootstrap sample but may have higher variance due to the nature of resampling with replacement.

Summary of Differences between Cross-Validation and Bootstrapping

Aspect	Cross-Validation	Bootstrapping
Definition	Splits data into k subsets (folds) for training and validation.	Samples data with replacement to create multiple bootstrap datasets.
Purpose	Estimate model performance and generalize to unseen data.	Estimate the variability of a statistic or model performance.
Process	1. Split data into k folds. 2. Train on $k-1$ folds, validate on the remaining fold.	1. Randomly sample data with replacement. 2. Repeat to create multiple bootstrap samples.

Aspect	Cross-Validation	Bootstrapping
	3. Repeat process k times (each fold serves as validation once).	3. Evaluate model on each bootstrap sample.
Advantages	1. Helps in model selection and tuning. 2. Reduces overfitting by validating on unseen data.	1. Captures uncertainty in model estimates 2. Useful for assessing bias and variance.
Disadvantages	Computationally intensive for large k or datasets.	May overestimate performance due to sample similarity.
Applicability	Commonly used in model evaluation and selection.	Useful when dataset size is limited or unknown distribution.

When to Use Cross-Validation

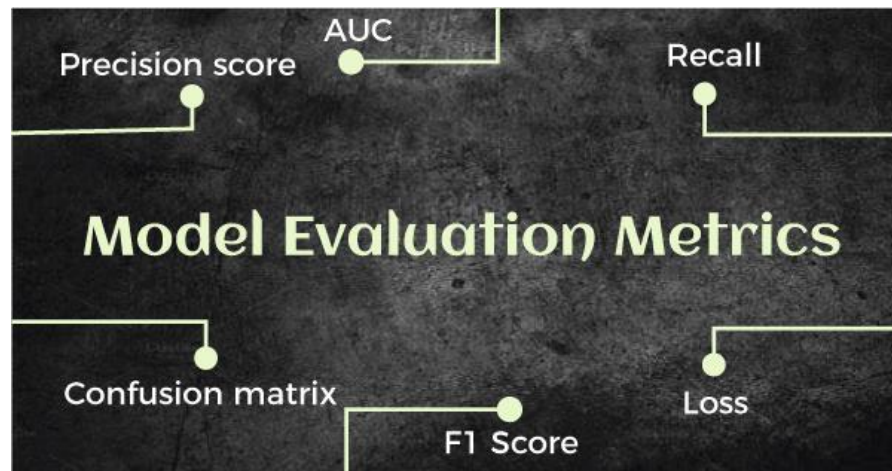
- **Model Comparison:** When comparing multiple models or algorithms.
- **Hyperparameter Tuning:** When tuning hyperparameters to find the best model configuration.
- **Balanced Datasets:** Works well when the dataset is balanced and large enough to be split into meaningful folds.

When to Use Bootstrapping

- **Small Datasets:** More effective for small datasets where splitting into multiple folds might not be feasible.
- **Variance Estimation:** When an estimate of the variability of the model performance is needed.
- **Uncertain Data:** When the dataset has significant noise or uncertainty.

PERFORMANCE METRICS IN MACHINE LEARNING

Evaluating the performance of a Machine learning model is one of the important steps while building an effective ML model. *To evaluate the performance or quality of the model, different metrics are used, and these metrics are known as performance metrics or evaluation metrics.* These performance metrics help us understand how well our model has performed for the given data. In this way, we can improve the model's performance by tuning the hyper-parameters. Each ML model aims to generalize well on unseen/new data, and performance metrics help determine how well the model generalizes on the new dataset.



In machine learning, each task or problem is divided into **classification** and **Regression**. Not all metrics can be used for all types of problems; hence, it is important to know and understand which metrics should be used. Different evaluation metrics are used for both Regression and Classification tasks. In this topic, we will discuss metrics used for classification and regression tasks.

1. Performance Metrics for Classification

In a classification problem, the category or classes of data is identified based on training data. The model learns from the given dataset and then classifies the new data into classes or groups based on the training. It predicts class labels as the output, such as *Yes or No*, *0 or 1*, *Spam or Not Spam*, etc. To evaluate the performance of a classification model, different metrics are used, and some of them are as follows:

- **Accuracy**
- **Confusion Matrix**
- **Precision**
- **Recall**
- **F-Score**
- **AUC(Area Under the Curve)-ROC**

I. Accuracy

The accuracy metric is one of the simplest Classification metrics to implement, and it can be determined as the number of correct predictions to the total number of predictions.

It can be formulated as:

$$\text{Accuracy} = \frac{\text{Number of Correct Predictions}}{\text{Total number of predictions}}$$

II. Confusion Matrix

A confusion matrix is a tabular representation of prediction outcomes of any binary classifier, which is used to describe the performance of the classification model on a set of test data when true values are known.

The confusion matrix is simple to implement, but the terminologies used in this matrix might be confusing for beginners.

A typical confusion matrix for a binary classifier looks like the below image(However, it can be extended to use for classifiers with more than two classes).

n=165	Predicted: NO	Predicted: YES
Actual: NO	50	10
Actual: YES	5	100

We can determine the following from the above matrix:

- In the matrix, columns are for the prediction values, and rows specify the Actual values. Here Actual and prediction give two possible classes, Yes or No. So, if we are predicting the presence of a disease in a patient, the Prediction column with Yes means, Patient has the disease, and for NO, the Patient doesn't have the disease.
- In this example, the total number of predictions are 165, out of which 110 time predicted yes, whereas 55 times predicted No.
- However, in reality, 60 cases in which patients don't have the disease, whereas 105 cases in which patients have the disease.

In general, the table is divided into four terminologies, which are as follows:

1. True Positive(TP): In this case, the prediction outcome is true, and it is true in reality, also.
2. True Negative(TN): in this case, the prediction outcome is false, and it is false in reality, also.
3. False Positive(FP): In this case, prediction outcomes are true, but they are false in actuality.
4. False Negative(FN): In this case, predictions are false, and they are true in actuality.

III. Precision

The precision metric is used to overcome the limitation of Accuracy. The precision determines the proportion of positive prediction that was actually correct. It can be calculated as the True Positive or predictions that are actually true to the total positive predictions (True Positive and False Positive).

$$Precision = \frac{TP}{(TP + FP)}$$

IV. Recall or Sensitivity

It is also similar to the Precision metric; however, it aims to calculate the proportion of actual positive that was identified incorrectly. It can be calculated as True Positive or predictions that are actually true to the total number of positives, either correctly predicted as positive or incorrectly predicted as negative (true Positive and false negative).

The formula for calculating Recall is given below:

$$\text{Recall} = \frac{TP}{TP + FN}$$

V. F-Scores

F-score or F1 Score is a metric to evaluate a binary classification model on the basis of predictions that are made for the positive class. It is calculated with the help of Precision and Recall. It is a type of single score that represents both Precision and Recall. So, *the F1 Score can be calculated as the harmonic mean of both precision and Recall, assigning equal weight to each of them.*

The formula for calculating the F1 score is given below:

$$F1 - score = 2 * \frac{\text{precision} * \text{recall}}{\text{precision} + \text{recall}}$$

VI. AUC-ROC

Sometimes we need to visualize the performance of the classification model on charts; then, we can use the AUC-ROC curve. It is one of the popular and important metrics for evaluating the performance of the classification model.

Firstly, let's understand ROC (Receiver Operating Characteristic curve) curve. *ROC represents a graph to show the performance of a classification model at different threshold levels.* The curve is plotted between two parameters, which are:

- **True Positive Rate**
- **False Positive Rate**

TPR or true Positive rate is a synonym for Recall, hence can be calculated as:

$$TPR = \frac{TP}{TP + FN}$$

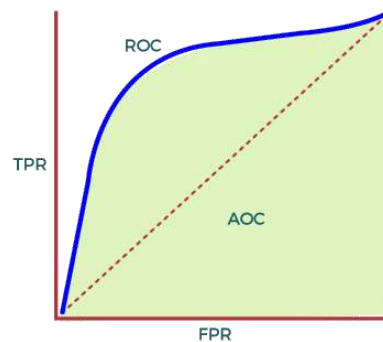
FPR or False Positive Rate can be calculated as:

$$FPR = \frac{FP}{FP + TN}$$

To calculate value at any point in a ROC curve, we can evaluate a logistic regression model multiple times with different classification thresholds, but this would not be much efficient. So, for this, one efficient method is used, which is known as AUC.

AUC: Area Under the ROC curve

AUC is known for **Area Under the ROC curve**. As its name suggests, AUC calculates the two-dimensional area under the entire ROC curve, as shown below image:



AUC calculates the performance across all the thresholds and provides an aggregate measure. The value of AUC ranges from 0 to 1. It means a model with 100% wrong prediction will have an AUC of 0.0, whereas models with 100% correct predictions will have an AUC of 1.0.

2. Performance Metrics for Regression

Regression is a supervised learning technique that aims to find the relationships between the dependent and independent variables. A predictive regression model predicts a numeric or discrete value. The metrics used for regression are different from the classification metrics. It means we cannot use the Accuracy metric (explained above) to evaluate a regression model; instead, the performance of a Regression model is reported as errors in the prediction. Following are the popular metrics that are used to evaluate the performance of Regression models.

- **Mean Absolute Error**
- **Mean Squared Error**
- **R2 Score**
- **Adjusted R2**

I. Mean Absolute Error (MAE)

Mean Absolute Error or MAE is one of the simplest metrics, which measures the absolute difference between actual and predicted values, where absolute means taking a number as Positive.

To understand MAE, let's take an example of Linear Regression, where the model draws a best fit line between dependent and independent variables. To measure the MAE or error in prediction, we need to calculate the difference between actual values and predicted values. But in order to find the absolute error for the complete dataset, we need to find the mean absolute of the complete dataset.

The below formula is used to calculate MAE:

$$MAE = 1/N \sum |Y - Y'|$$

Here,

Y is the Actual outcome, Y' is the predicted outcome, and N is the total number of data points.

II. Mean Squared Error

Mean Squared error or MSE is one of the most suitable metrics for Regression evaluation. It measures the average of the Squared difference between predicted values and the actual value given by the model.

The formula for calculating MSE is given below:

$$MSE = 1/N \sum (Y - Y')^2$$

Here,

Y is the Actual outcome, Y' is the predicted outcome, and N is the total number of data points.

III. R Squared Score

R squared error is also known as Coefficient of Determination, which is another popular metric used for Regression model evaluation. The R-squared metric enables us to compare our model with a constant baseline to determine the performance of the model. To select the constant baseline, we need to take the mean of the data and draw the line at the mean.

The R squared score will always be less than or equal to 1 without concerning if the values are too large or small.

$$R^2 = 1 - \frac{MSE(Model)}{MSE(Baseline)}$$

IV. Adjusted R Squared

Adjusted R squared, as the name suggests, is the improved version of R squared error. R square has a limitation of improvement of a score on increasing the terms, even though the model is not improving, and it may mislead the data scientists.

To overcome the issue of R square, adjusted R squared is used, which will always show a lower value than R². It is because it adjusts the values of increasing predictors and only shows improvement if there is a real improvement.

We can calculate the adjusted R squared as follows:

$$R_a^2 = 1 - \left[\left(\frac{n-1}{n-k-1} \right) \times (1 - R^2) \right]$$

Here,

n is the number of observations

k denotes the number of independent variables

and R_a² denotes the adjusted R²

ASSESSING A CLASSIFICATION ALGORITHM'S PERFORMANCE

We will discuss the case of classification error, but the same methodology applies for squared error in regression, log likelihoods in unsupervised learning, expected reward in reinforcement learning, and so on, as long as we can write the appropriate parametric form for the sampling distribution. We will also discuss nonparametric tests when no such parametric form can be found.

Binomial Test

- Let us start with the case where we have a single training set T and a single validation set V . We train our classifier on T and test it on V . We denote by p the probability that the classifier makes a misclassification error.
- We do not know p ; it is what we would like to estimate or test a hypothesis about.
- On the instance with index t from the validation set V , let us say x_t denotes the correctness of the classifier's decision: x_t is a 0/1 Bernoulli random variable that takes the value 1 when the classifier commits an error and 0 when the classifier is correct.
- The binomial random variable X denotes the total number of errors:

$$X = \sum_{t=1}^N x_t$$

We would like to test whether the error probability p is less than or equal to some value p_0 we specify:

$$H_0 : p \leq p_0 \text{ vs. } H_1 : p > p_0$$

If the probability of error is p , the probability that the classifier commits j errors out of N is

$$P\{X = j\} = \binom{N}{j} p^j (1 - p)^{N-j}$$

It is reasonable to reject $p \leq p_0$ if in such a case, the probability that binomial test we see $X = e$ errors or more is very unlikely. That is, the binomial test rejects the hypothesis if

$$P\{X \geq e\} = \sum_{x=e}^N \binom{N}{x} p_0^x (1-p_0)^{N-x} < \alpha \quad (19.10)$$

where α is the significance, for example, 0.05.

Approximate Normal Test

- If p is the probability of error, our point estimate is $\hat{p} = X/N$. Then, it is reasonable to reject the null hypothesis if $\hat{p}$ is much larger than p_0 .
- How large is large enough is given by the sampling distribution of $\hat{p}$ and the significance α . Because X is the sum of independent random variables from the same distribution, the central limit theorem states that for large N , X/N is approximately normal with mean p_0 and variance $p_0(1 - p_0)$. Then

$$X/N - p_0 / \sqrt{p_0(1 - p_0)} \sim Z \quad (19.11)$$

The approximate normal test rejects the null hypothesis if this value for $X = e$ is greater than z_α . $Z_{0.05}$ is 1.64. This approximation will work well as long as N is not too small and p is

not very close to 0 or 1; as a rule of thumb, we require $Np \geq 5$ and $N(1 - p) \geq 5$.

COMPARING TWO CLASSIFICATION ALGORITHMS

T-TEST

Paired t-Test A paired t-test is a statistical test used to compare the means of two related samples or matched pairs. It is used to test a hypothesis about the difference between the means of the two samples. It is based on the assumption that the differences between the pairs are normally distributed.

Dependent vs Independent Samples

There are two types of two-sample t-tests: dependent and independent.

In an **independent two-sample t-test** (also known as an unpaired t-test), the samples in the two groups being compared are unrelated. The samples are drawn from two different populations or groups of subjects, and the difference between the means of the two groups is calculated using the means and variances of the two separate samples. **A separate post covers the independent two-sample t-test.**

In a **dependent two-sample t-test** (also known as a **paired t-test**), the samples in the two groups being compared are related in some way. For example, the samples may be pairs of measurements taken on the same subjects or on subjects who are closely matched in some other way. In this case, the difference between the means of the two groups is calculated by taking the differences between the pairs of measurements and treating these differences as a single sample. **This post covers the paired t-test or dependent two-sample t-test.** The choice between a dependent or independent t-test depends on the nature of your data and the research question you are trying to answer.

Steps in Paired t Test

To conduct a paired t-test, the following steps are typically followed:

1. **Specify the null and alternative hypotheses.** The null hypothesis is usually that there is no difference between the means of the two samples, while the alternative hypothesis is that there is a difference between the means.
2. **Collect data for the two related samples or matched pairs.**
3. Calculate the **differences between the pairs** and **find the mean and standard deviation of the differences.**
4. **Calculate the test statistic**, which is the difference between the means of the two samples, divided by the standard error of the mean.
5. **Determine the critical value of the test statistic** based on the significance level (alpha) of the test and the degrees of freedom. The degrees of freedom are calculated as the sample size minus 1.

6. **Compare the calculated test statistic to the critical value** to determine whether to reject or fail to reject the null hypothesis. If the calculated test statistic exceeds the critical value, the null hypothesis is rejected, and the alternative hypothesis is accepted.

Conditions for Paired t-Test

To conduct a valid paired t-test, the following conditions must be met:

1. The samples must be related or matched in some way.
2. The differences between the pairs must be approximately normally distributed.
3. The population standard deviation of the differences is unknown, and the sample size is small (less than 30).

Typical Null and Alternate Hypothesis in Paired t-test

a) Two-Tail Test:

In a paired t-test, the null hypothesis is that there is no average difference between the two samples. This can be expressed as:

$$H_0: \mu_1 - \mu_2 = 0$$

where μ_1 is the mean of the first sample and μ_2 is the mean of the second sample.

The alternate hypothesis is the opposite of the null hypothesis and is that there is a difference between the means of the two samples. This can be expressed as:

$$H_a: \mu_1 - \mu_2 \neq 0$$

b) Left Tail Test:

A left-tailed hypothesis is one in which the mean of the first sample is less than the mean of the second sample. This can be expressed as:

$$H_0: \mu_1 - \mu_2 \geq 0$$

$$H_a: \mu_1 - \mu_2 < 0$$

c) Right Tail Test:

A right-tailed hypothesis is one in which the mean of the first sample is greater than the mean of the second sample. This can be expressed as:

$$H_0: \mu_1 - \mu_2 \leq 0$$

$$H_a: \mu_1 - \mu_2 > 0$$

Calculating Test Statistic

The t-score represents the number of standard errors that the difference between the means of the two samples from zero. It is used to determine whether the difference between the means is statistically significant.

The formula for calculating the t-score in a paired t-test is as follows:

$$t = \frac{\bar{d}}{\frac{s_d}{\sqrt{n}}}$$

where:

- $\bar{d}$ is the mean of the difference scores
- s_d is the standard deviation of the difference scores
- n is the number of pairs of observations

QG

Paired t Tests

$H_0: \mu_{\text{before}} = \mu_{\text{after}}$
 $H_a: \mu_{\text{before}} \neq \mu_{\text{after}}$

$$t = \frac{\bar{d}}{s/\sqrt{n}}$$

❖ Example: Before and after medicine BP was measured. Is there a **difference** at 95% confidence level?

❖ $\bar{d} = -1.4$, $s = 4.56$, $n = 5$

❖ $t_{\text{cal.}} = 1.4/2.04 = -0.69$

Patient	Before	After	difference
1	120	122	-2
2	122	120	2
3	143	141	2
4	100	109	-9
5	109	109	0

Paired t Test

McNemar's Test

Given a training set and a validation set, we use two algorithms to train two classifiers on the training set and test them on the validation set and compute their errors. A contingency table, like the one shown here, is an array of natural numbers in matrix form representing counts, or frequencies:

e_{00} : Number of examples misclassified by both	e_{01} : Number of examples misclassified by 1 but not 2
e_{10} : Number of examples misclassified by 2 but not 1	e_{11} : Number of examples correctly classified by both

Under the null hypothesis that the classification algorithms have the same error

rate, we expect $e_{01} = e_{10}$ and these to be equal to $(e_{01} + e_{10})/2$. We have the chi-square statistic with one degree of freedom

$$(|e_{01} - e_{10}| - 1)^2 / (e_{01} + e_{10}) \sim \chi^2_1$$

and McNemar's test rejects the hypothesis that the two classification algorithms have the same error rate at significance level α if this value is greater than $\chi^2_{\alpha,1}$. For $\alpha = 0.05$, $\chi^2_{0.05,1} = 3.84$

K-Fold Cross-Validated Paired t Test

This set uses K-fold cross-validation to get K training/validation set pairs. We use the two classification algorithms to train on the training sets T_i , $i = 1, \dots, K$, and test on the validation sets V_i . The error percentages of the classifiers on the validation sets are recorded as p_{1i} and p_{2i} .

If the two classification algorithms have the same error rate, then we expect them to have the same mean, or equivalently, that the difference of their means is 0. The difference in error rates on fold i is $p_i = p_{1i} - p_{2i}$. This is a paired test; that is, for each i , both algorithms see the same training and validation sets. When this is done K times, we have a distribution of p_i containing K points. Given that p_{1i} and p_{2i} are both (approximately) normal, their difference p_i is also normal. The null hypothesis is that this distribution has 0 mean:

$$H_0 : \mu = 0 \text{ vs. } H_1 : \mu \neq 0$$

We define

$$\bar{m} = \sum_{i=1}^K p_i / K, S^2 = \sum_{i=1}^K (p_i - \bar{m})^2 / (K - 1)$$

Under the null hypothesis that $\mu = 0$, we have a statistic that is t distributed with $K - 1$ degrees of freedom:

$$\sqrt{K}(\bar{m} - 0) / S = \sqrt{K} \cdot \bar{m} / S \sim t_{K-1} \quad (19.14)$$

- Thus the K-fold cv paired t test rejects the hypothesis that two classification algorithms have the same error rate at significance level α if this value is outside the interval $(-t_{\alpha/2, K-1}, t_{\alpha/2, K-1})$. $t_{0.025, 9} = 2.26$ and $t_{0.025, 29} = 2.05$.
- If we want to test whether the first algorithm has less error than the second, we need a one-sided hypothesis and use a one-tailed test:

$$H_0 : \mu \geq 0 \text{ vs. } H_1 : \mu < 0$$

If the test rejects, our claim that the first one has significantly less error is supported.

K-Fold Cross Validation

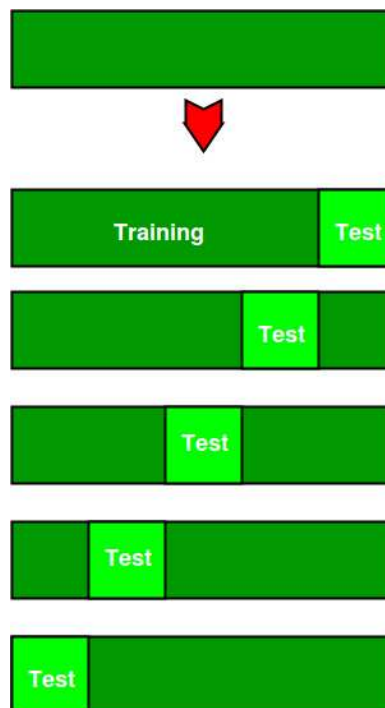
In K-Fold Cross Validation, we split the dataset into k number of subsets (known as folds) then we perform training on the all the subsets but leave one($k-1$) subset for the evaluation of the trained model. In this method, we iterate k times with a different subset reserved for testing purpose each time.

Note: It is always suggested that the value of k should be 10 as the lower value of k takes towards validation and higher value of k leads to LOOCV method.

Example of K Fold Cross Validation

The diagram below shows an example of the training subsets and evaluation subsets generated in k -fold cross-validation. Here, we have total 25 instances. In first iteration we use the first 20 percent of data for evaluation, and the remaining 80 percent for training ([1-5] testing and [5-25] training) while in the second iteration we use the second subset of 20 percent for evaluation,

and the remaining three subsets of the data for training ([5-10] testing and [1-5 and 10-25] training), and so on.



Advantages:

5. Overcoming Overfitting: Cross validation helps to prevent overfitting by providing a more robust estimate of the model's performance on unseen data.
6. Model Selection: Cross validation can be used to compare different models and select the one that performs the best on average.
7. Hyperparameter tuning: Cross validation can be used to optimize the hyperparameters of a model, such as the regularization parameter, by selecting the values that result in the best performance on the validation set.

8. **Data Efficient:** Cross validation allows the use of all the available data for both training and validation, making it a more data-efficient method compared to traditional validation techniques.

Disadvantages:

4. **Computationally Expensive:** Cross validation can be computationally expensive, especially when the number of folds is large or when the model is complex and requires a long time to train.
5. **Time-Consuming:** Cross validation can be time-consuming, especially when there are many hyperparameters to tune or when multiple models need to be compared.
6. **Bias-Variance Tradeoff:** The choice of the number of folds in cross validation can impact the bias-variance tradeoff, i.e., too few folds may result in high variance, while too many folds may result in high bias.